



Project Name : Elrehla

Doc	3D	Keyboard and mouse	Collision	Camera	Texture	Lighting

Mark	Name	Section
	مصطفى محمد عبد الستار ابراهيم	6 CS
	مصطفى ياسر السيد	6 CS
	معاذ عبدالمنعم عبدالعليم	6 CS
	مريم جمال رشدي	6 CS
	مريم محمد محمد	6 CS
	هابي هدراميشيل	6 CS

Faculty of Computer Science and AI

Dr. Rasha Orban

Eng. Ahmed Megahed

1. Project Concept

El-Rehla is a 3D state-driven maze engine built in C++. It procedurally generates a 3D environment from a static 2D Integer Matrix. The project focuses on transforming grid-based logic into an immersive spatial experience, utilizing a "Player vs. Environment" architecture where the goal is to navigate to a specific coordinate to trigger a victory state.

2. Core Programming Logic

- **Grid-to-World Mapping:** The engine iterates through a ROWS x COLS array. If a cell contains 1, the code calculates 3D coordinates and renders a textured cube (GL_QUADS) at that specific world position.
- **Sampling Collision System:** Instead of basic bounding boxes, the code samples 8 points around the player's radius. It converts these 3D world coordinates back into array indices to check if the next move is valid (path 0) or blocked (wall 1).
- **Dynamic Viewport Logic:** Using gluLookAt, the code manages two distinct camera states. It handles coordinate transformations between a third-person follow-camera and a rotating orbital camera based on user input.

3. Technical Stack

- **Graphics API:** OpenGL & GLUT for the rendering pipeline, depth testing, and double-buffering.
- **Texture Engine:** stb_image for loading and binding .jpg and .png textures to 3D primitives.
- **Lighting Model:** GL_LIGHT0 implementation with ambient and diffuse properties to provide depth and volume to the maze.
- **Real-time Math:** Use of <cmath> for sine-wave animations (diamond bobbing) and Euclidean distance formulas for objective detection.

4. State & Resource Management

- **UI Overlay:** When the isWinner state is true, the code switches from Perspective to Orthographic projection (glOrtho) to render a 2D "Winner" screen.
- **Frame-Rate Control:** Uses glutTimerFunc to ensure consistent animation speeds and physics updates across different hardware.

How to play

1. The Objective

Your mission is to navigate the **Pac-Man** through a complex 25x25 maze. The goal is to reach the **Spinning Blue Diamond** located at the exit of the maze. Once you touch the diamond, the "Winner" screen will appear, marking the successful completion of your journey.

2. Basic Movement (Keyboard)

The player movement is designed to be responsive and intuitive. Use the following keys to navigate:

Key	Action
Up Arrow	Move Forward (North)
Down Arrow	Move Backward (South)
Left Arrow	Move Left (West)
Right Arrow	Move Right (East)

Note: Pac-Man will automatically rotate his body to face the direction you are moving.

3. Camera Modes & Perspective

The game offers two different ways to view the world. You can switch between them at any time to get a better view of the maze:

- **Toggle View (V Key):** Press 'V' to switch between **Third-Person Follow** and **Orbital View**.
- **Rotate Camera (A / D Keys):** While in **Orbital View**, use 'A' to rotate the camera left and 'D' to rotate it right. This helps you scout the maze paths from different angles.

4. Game Environment & Boundaries

- **Collision System:** You cannot pass through the maze walls. The game uses a radius-based collision check, so if you get too close to a wall, Pac-Man will stop moving.
- **The Goal (Diamond):** Look for the glowing, rotating blue diamond. It bobs up and down to make it easier to spot from a distance.
- **Full-Screen Mode (F1):** Press '**F1**' to toggle between Windowed mode and Full-Screen mode for a more immersive experience.
- **Exit (Esc):** Press the '**Escape**' key at any time to close the game.

5. Winning the Game

To win, you must find the specific opening in the outer walls that leads to the Diamond. The collision with the Diamond is detected automatically once you are within **1.5 units** of its center.